Chapter 5:Advanced SQL

Programmable SQL: Function, Procedure and Trigger

Abu Raihan Mostofa Kamal

Professor, CSE Department
Islamic University of Technology (IUT)
Gazipur-1704 Bangladesh
raihan.kamal@iut-dhaka.edu
+8801843925543

January 27, 2026

Based on Chapter 5: Main text book

Chapter Outline

PL/SQL Block and conditional statement

Conditional Construct, Iteration

PL/SQL Block

Block: Smallest unit of code

In PL/SQL, as in most other procedural languages, **the smallest meaningful grouping of code** is known as a block. A block is a unit of code that provides **execution and scoping boundaries for variable declarations and exception handling**.

Blocks: Anonymous

Blocks: Anonymous

It has no Name

It cannot be called by any other blockit doesnt have a handle for reference. Instead, anonymous blocks serve as containers that execute PL/SQL statements. It is **not stored** in the database and **can not be reused**. (one-time execution).

```
BEGIN

DBMS_OUTPUT.put_line ('Hello World!');

END;
```

Block: Named

It has Name

While anonymous PL/SQL blocks are indispensable, the majority of code you write will be in named blocks. **Examples:** Functions and Procedure. It is stored in the database and can be reused.

Block: Equivalence in Oracle and Postgre SQL

In Oracle, you start an anonymous block with **DECLARE** or **BEGIN**. In PostgreSQL, you must use the **DO** keyword. By default, **DO** executes code in the plpgsql language.

Oracle

```
DECLARE
v_count NUMBER;
BEGIN
SELECT count(*) INTO
    v_count FROM
    transactions;
DBMS_OUTPUT.PUT_LINE('Total:
    ' || v_count);
END;
```

Postgre SQL

```
DO $$
DECLARE
v_count INTEGER;
BEGIN
SELECT count(*) INTO
    v_count FROM
    transactions;
RAISE NOTICE 'Total: %',
    v_count;
END $$;
```

Conditional Statement: Equivalence in Oracle and Postgre SQL

Oracle

```
DECLARE
v_balance NUMBER := 150;
BEGIN
  IF v_balance > 100 THEN
  DBMS_OUTPUT.PUT_LINE('High Balance');
  ELSIF v_balance > 50 THEN
  DBMS_OUTPUT.PUT_LINE('Medium Balance');
  ELSE
  DBMS_OUTPUT.PUT_LINE('Low Balance');
  END IF;
END;
```

Postgre SQL

```
DO $$
DECLARE
v_balance NUMERIC := 150;
BEGIN
  IF v_balance > 100 THEN
  RAISE NOTICE 'High Balance';
  ELSIF v_balance > 50 THEN
  RAISE NOTICE 'Medium Balance';
  ELSE
  RAISE NOTICE 'Low Balance';
  END IF;
END $$;
```

Iteration: Equivalence in Oracle and Postgre SQL

Oracle

```
BEGIN
FOR i IN 1..5 LOOP
DBMS_OUTPUT.PUT_LINE('Iteration:
    ' || i);
END LOOP;
END;
```

Postgre SQL

```
DO $$
BEGIN
FOR i IN 1..5 LOOP
RAISE NOTICE
    'Iteration: %', i;
END LOOP;
END $$;
```

Function

Modular Code using Function

- More reusable
- More manageable
- More readable

Function

Modular Code using Function

- More reusable
- More manageable
- More readable

Functions (Cont.): Equivalence in Oracle and Postgre SQL

Oracle

```
CREATE OR REPLACE FUNCTION
    calculate_tax(p_amount
    NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN p_amount * 0.10;
END;
/
```

Postgre SQL

```
CREATE OR REPLACE FUNCTION
    calculate_tax(p_amount
    NUMERIC)
RETURNS NUMERIC
AS $$
BEGIN
RETURN p_amount * 0.10;
END;
$$ LANGUAGE plpgsql;
```

A few points about Functions

- IN parameter (like conventional parameter), OUT parameter (not used here. used for writing the result back to the caller)
- No datatype precision is allowed in parameter and return type (**...return varchar2(20)** is not valid) **applicable both in Oracle and Postgre**
- Function Name and parameters should be **self-explanatory**. (do not use abc as function name, x, y , z as parameters)
- Remember: in Oracle **Type Strictness** is relaxed while in Postgre it is very strict often needs manual casting.

Function: A simple example

```
CREATE OR REPLACE FUNCTION get_total_sales
RETURN NUMBER
IS
v_total_sales NUMBER := 0;
BEGIN
-- get total sales
SELECT SUM(unit_price * quantity)
INTO l_total_sales
FROM order_items
INNER JOIN orders USING (order_id)
WHERE status = 'Shipped';


-- return the total sales
RETURN v_total_sales;
END;
```

Calling a function

There are 3 general options:

1. It can be readily tested by a fixed parameter from dual:
   ```
   select get_total('CSE') from dual;
   ```

Calling a function

There are 3 general options:

1. It can be readily tested by a fixed parameter from dual:
   ```
   select get_total('CSE') from dual;
   ```
2. You can test it by passing it as a parameter in put_line function:
   ```
   DBMS_OUTPUT.PUT_LINE('Sales: ' || get_total_sales);
   ```

Calling a function

There are 3 general options:

1. It can be readily tested by a fixed parameter from dual:
   ```
   select get_total('CSE') from dual;
   ```
2. You can test it by passing it as a parameter in put_line function:
   ```
   DBMS_OUTPUT.PUT_LINE('Sales: ' || get_total_sales);
   ```
3. It can be selected from a table values, the parameter values can be passed dynamically in the select statement.
   ```
   select id, get_cgpa(sid) from students where dept='CSE';
   ```

DECODE function

```
DECODE(expr, search, result
[, search, result ]...
[, default ]
)
```

DECODE()

DECODE compares expr to each search value one by one. If expr is equal to a search, then Oracle Database returns the corresponding result. If no match is found, then Oracle returns default. If default is omitted, then Oracle returns null.

DECODE by example

```
SELECT product_id,
DECODE (warehouse_id, 1, 'Southlake',
2, 'San Francisco',
3, 'New Jersey',
4, 'Seattle',
'Non domestic')
"Location of inventory" FROM inventories
WHERE product_id < 100;
```

RANK() function

- The RANK() function is **an analytic function** that calculates the rank of a value in a set of values.

RANK() function

- The RANK() function is **an analytic function** that calculates the rank of a value in a set of values.
- The RANK() function returns the same rank for the rows with the same values. It adds the number of tied rows to the tied rank to calculate the next rank. Therefore, the ranks **may not be consecutive numbers**.

RANK() function

- The **RANK()** function is **an analytic function** that calculates the rank of a value in a set of values.
- The RANK() function returns the same rank for the rows with the same values. It adds the number of tied rows to the tied rank to calculate the next rank. Therefore, the ranks **may not be consecutive numbers**.
- The RANK() function is useful for **top-N** and **bottom-N** queries.

RANK() Syntax and Example:

```
RANK()
OVER ([ query_partition_clause ] order_by_clause)

Example:

select sid,name,cgpa,
rank() over (order by cgpa desc) position
from students;
```

Triggers (Elementary)

- A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.

Triggers (Elementary)

- A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:

Triggers (Elementary)

- A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
  - Specify the conditions under which the trigger is to be executed.

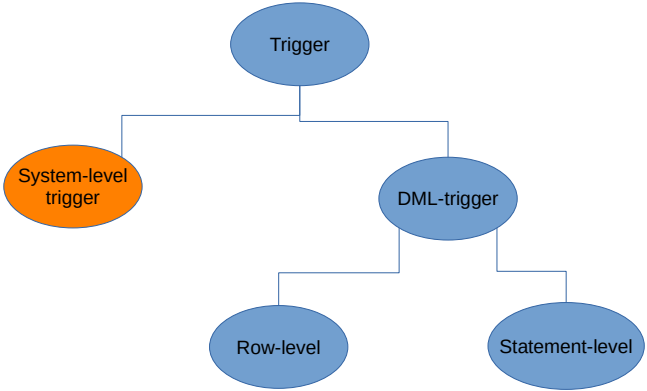
Triggers (Elementary)

- A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
  - Specify the conditions under which the trigger is to be executed.
  - Specify the actions to be taken when the trigger executes.

Triggers (Elementary)

- A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
  - Specify the conditions under which the trigger is to be executed.
  - Specify the actions to be taken when the trigger executes.
- Since trigger is event-oriented, it is not explicitly called like functions rather it automatically fires wherever the condition is met.

Trigger : Broad Classification



Row-level Trigger

Some examples:

- Whenever an account holder withdraws money, his balance must be greater than the withdrawn amount.

Row-level Trigger

Some examples:

- Whenever an account holder withdraws money, his balance must be greater than the withdrawn amount.
- Any student is admitted with basic information, his/her Student ID will be generated automatically.

Row-level Trigger

Some examples:

- Whenever an account holder withdraws money, his balance must be greater than the withdrawn amount.
- Any student is admitted with basic information, his/her Student ID will be generated automatically.
- Whenever any update (1 or more rows affected) occurs in a specific table , the time of update should be recorded without human intervention.

Row-level Trigger

Some examples:

- Whenever an account holder withdraws money, his balance must be greater than the withdrawn amount.
- Any student is admitted with basic information, his/her Student ID will be generated automatically.
- Whenever any update (1 or more rows affected) occurs in a specific table , the time of update should be recorded without human intervention.

Row-level Trigger

Some examples:

- Whenever an account holder withdraws money, his balance must be greater than the withdrawn amount.
- Any student is admitted with basic information, his/her Student ID will be generated automatically.
- Whenever any update (1 or more rows affected) occurs in a specific table , the time of update should be recorded without human intervention.

So, they have one in common: **it is executed for each possible record's change.**

Row-level Trigger (Cont.), Syntax

Indicator: **for each row** clause

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE | AFTER
INSERT OR DELETE OR UPDATE OF column1, column2,
ON table_name
FOR EACH ROW
REFERENCING OLD AS old_name
NEW AS new_name
WHEN (condition)
DECLARE

BEGIN

EXCEPTION

END;
```

Statement-level Trigger

It works exactly in the same way as except it is executed only once no matter how many records are affected by the DML statement.

Syntax: Just omit the **for each row** option.